SCALER School of Technology × Priyansh Saxena

Model Serving: REST, gRPC, and Streaming

Prediction Modes ▪ Latency & Throughput ▪ Deployment Strategies

Priyansh Saxena

Scaler School of Technology | ML System Design and LLMOps

Week 2 — Session 2/2

ROADMAP

Agenda

1. The Reality of Model Deployment
2. Batch vs. Online Prediction
3. Latency, Throughput, and SLOs
4. Serving Patterns and Deployment Strategies

TOC

1

The Reality of Model Deployment

1 Deployment Myth 1 — “We only have a couple of models”

The myth

- “We deploy 1 or 2 models”
- “Model management is not a big deal”
- “One inference server is enough”

The reality

- **Uber**: thousands of models in production
- **Google**: thousands training *concurrently*
- **Netflix**: ML for 20+ tasks

Netflix uses ML for:

- Content valuation & acquisition
- Churn prediction & retention
- Fraud detection
- Machine translation (subtitles)
- Content tagging & search
- Personalised thumbnails
- . . . and 14+ more tasks

A ride-sharing app alone needs

Ride demand, driver availability, ETA, dynamic pricing, fraud detection, churn. 20 countries $\times$ 10 models each $=$ **200 models**.

1 Deployment Myth 2 & 3 — Performance and Updates

Myth 2: "Do nothing, performance stays the same"

- Software ages poorly
- ML models *age worse*
- A model performs **best immediately after training**
- Data distribution shifts: user behaviour, world events, seasonality
- No retraining = silent degradation

Myth 3: "We won't need to update much"

Wrong question: "How often *should* I update?"

Right question: "How often *can* I update?"

Industry update cadences

- **Weibo**: some models updated every **10 minutes**
- **Netflix**: thousands of pushes per day
- **AWS**: new deployment every **11.7 seconds**

1 Deployment Myth 4 — “Scale is only for FAANG”

The myth

“I work at a startup. Scale does not apply to me.”

Why this is wrong:

- Most ML engineers work at companies with 100+ employees
- A company of 100 employees likely has a meaningful number of users
- Your ML model is running *per request*, not per company
- 10,000 users $\times$ 5 predictions/day = 50K daily inferences
- Add peak traffic spikes and the problem becomes real fast

What scale concerns actually are

- **Latency under load**: does p99 stay $<$100ms at 10$\times$ traffic?
- **Cost efficiency**: GPU utilisation at off-peak hours
- **Reliability**: what happens when one replica crashes?
- **Version management**: which model version is running where?

“An ML engineer should care about scale.”
— Chip Huyen, Ch. 7

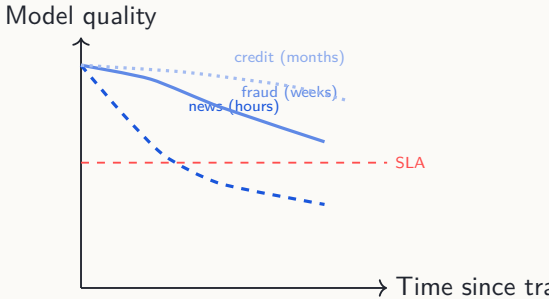
1 The Degradation Curve — Models Age Poorly

Why performance degrades

- **Data drift**: input distribution $P(X)$ shifts
- **Concept drift**: relationship $P(Y|X)$ changes
- **Label shift**: output distribution $P(Y)$ changes
- User behaviour changes seasonally
- Product features change, creating new patterns

Domain-specific decay rates

- News ranking: degrades in **hours**
- Fraud detection: degrades in **days–weeks**
- Credit scoring: degrades over **months**
- Medical imaging: may be stable for **years**



2

Batch vs. Online Prediction

2 Two Fundamental Prediction Modes

Online prediction (on-demand)

- Generated and returned *as soon as request arrives*
- Also called **synchronous** or **real-time** prediction
- User sends input → model computes → response returned
- Requests via REST API or gRPC

Example: You type an English sentence. Google Translate returns French immediately.

Batch prediction (pre-computed)

- Generated *periodically*, stored, retrieved when needed
- Also called **asynchronous** prediction
- Predictions exist before the user asks for them
- Stored in data warehouse / cache

Example: Netflix generates recommendations every 4 hours for all users. Fetched when you log in.

Key distinction

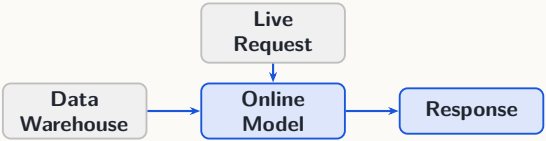
Online prediction waits for a request. Batch prediction runs ahead of time. The right choice depends on your latency requirement and data freshness needs.

2 Three Prediction Modes in Practice

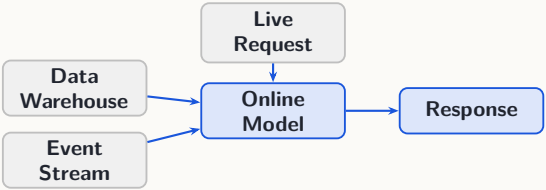
Mode 1: Batch prediction (batch features only)



Mode 2: Online prediction (batch features only)



Mode 3: Streaming prediction (batch + streaming features)



2 DoorDash: When You Need Both Feature Types

Problem: Predict delivery time accurately

- User places order at a restaurant
- Need an ETA *right now*
- ETA depends on: restaurant prep time + driver availability + traffic

Batch features (historical)

- Mean preparation time of this restaurant
- Average driver availability in this area
- Historical traffic patterns for this route

Source: data warehouse, computed overnight

Streaming features (real-time)

- How many orders at this restaurant in the **last 10 minutes**?
- How many drivers available **right now**?
- Any unusual traffic **at this moment**?

Source: event stream, computed in real time

Important clarification

Streaming features $\neq$ Online features
Online features = any feature used in online prediction (including pre-computed batch features).
Streaming features = computed exclusively from live event streams.

2 Batch vs. Online — Trade-offs

Dimension	Batch prediction	Online prediction
Latency	Not user-facing; latency irrelevant	User is waiting; latency critical (<500ms typical)
Throughput	High — process millions at once	Limited by server capacity per request
Freshness	Stale by design — hours or days old	Fresh — computed at request time
Feature complexity	Can use slow, expensive features	Must use features retrievable in milliseconds
Infrastructure	Spark, Airflow, scheduled jobs	REST/gRPC servers, load balancers, caches
When to use	Weekly recs, monthly churn scores	Fraud check, search ranking, chatbots

5-Minute Break

Your model can wait. You, however, need a break.

Stand up, stretch, grab some water. See you in five.

3

Latency, Throughput, and SLOs

3 Latency vs. Throughput — The Core Trade-off

Latency

- Time from *request sent* to *response received*
- NOT training time — that is a different metric
- Affected by: model size, feature lookup time, preprocessing, network
- What users feel

Throughput

- Queries processed per unit time (QPS)
- What determines your server cost
- Throughput $\neq 1/\text{latency}$ when batching

The batching paradox

- One query at a time: higher latency = lower throughput
- With batching: you can *increase* throughput by *accepting* slightly higher latency

Example:
Process 50 queries in a 20ms batch:
Throughput $= 50/0.020 =$ **2,500 QPS**
vs. 1 query in 2ms:
Throughput $= 1/0.002 =$ **500 QPS**

3 Why Percentiles, Not Means

10 requests, measured latencies (ms):

[100, 102, 100, 100, 99, 104, 110, 90, **3000**, 95]

- **Mean**: $\approx$ 390 ms (misleading!)
- **p50 (median)**: $\approx$ 100 ms
- **p95**: $\approx$ 110 ms
- **p99**: $\approx$ 3000 ms

The mean is dominated by the **3-second outlier**.
9 out of 10 users had a fine experience.
1 in 10 waited 3 seconds.

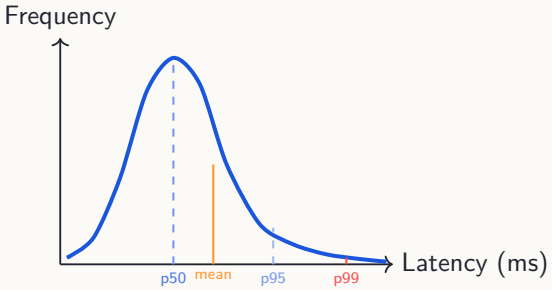
Why p99 matters most

- p99 = the 1% of users with the worst experience
- At 1M QPS, that is 10,000 users/second experiencing a broken product
- Large companies often target **p99.9**
- Mean-based SLAs can hide catastrophic tail latency

Real-world impact

Akamai (2017): 100ms delay $\rightarrow$ 7% drop in conversion.
Booking.com (2019): $+0.5\%$ latency $\rightarrow$ -0.5% conversion.

3 Latency Distribution — Reading the Numbers



How to set SLOs

1. Understand your user expectations
2. Measure current distribution
3. Set *realistic* targets at multiple percentiles

Common ML SLOs

- **p50 latency**: typical user experience target
- **p95 latency**: most users are served this well
- **p99 latency**: worst 1% — often the SLA threshold
- **Throughput**: minimum QPS under peak load
- **Availability**: uptime % (99.9% = 8.7 hrs downtime/year)
- **Error rate**: $<0.1\%$ failures

3 Quiz 1

Q1 — Fraud Detection Serving Mode

A fraud detection model must flag suspicious transactions in **under 50ms**. Every credit card transaction triggers a check. Transaction volume: 10,000 per second at peak.

Which prediction mode should you use? What are the main engineering challenges?

Q2 — Diagnosing a Latency Problem

Your model's **average** latency is 200ms, which your team considers fine. Your on-call engineer gets paged at 2am. Users are complaining. You look at the metrics and see: p99 latency = 4,000ms.

What does this tell you? What would you investigate first?

3 Quiz 1 — Answers

A1 — Fraud Detection Serving Mode

Mode: Online (synchronous) prediction. The 50ms SLA requires a decision before the transaction completes — batch prediction is impossible.

Engineering challenges: (1) Feature lookup latency — Redis online store keeps lookups under 5ms. (2) At 10K QPS, horizontally scale the serving fleet with load balancing. (3) p99 latency — one slow GPU call breaches the SLA; add request timeouts and fallback rules.

A2 — Diagnosing a Latency Problem

What it tells you: 99% of requests are fine, but 1% take 4 seconds. This is a **tail latency problem** — a long-tail event, not a general slowdown.

Investigate first: (1) Is p99 correlated with a specific feature lookup (slow DB query on rare users)? (2) Is there a cold-start problem (first request loads the model)? (3) Is a downstream service timing out on 1% of requests? Check per-component latency breakdowns in your tracing tool.

4

Serving Patterns and Deployment Strategies

4 Four Serving Patterns

Pattern	Use when	Examples
Synchronous REST	User is waiting; latency < 1 s; single item; simple ops	Fraud check at checkout, spam filter, search ranking
Asynchronous batch	High volume; no real-time need; cost-sensitive; overnight or hourly	Weekly recommendations, monthly churn scores, demand forecasts
Streaming (Kafka/Flink)	Continuous events; sub-second freshness; stateful aggregation	Real-time fraud, dynamic pricing, live anomaly detection
Edge / on-device	Privacy constraints; offline use; ultra-low latency $<$10 ms	Face unlock, keyword spotting, autonomous vehicle perception

4 Shadow, Canary, and Blue-Green Deployment

Shadow deployment

- New model runs in *parallel*
- Does **not** serve real traffic
- Predictions logged and compared offline
- Safe — zero user impact
- Cost: doubles compute

Best for: high-risk models, first validation step

Canary deployment

- 5–10% of traffic → new model
- Monitor real metrics on real users
- Gradually increase if stable
- Roll back fast if metrics drop

Best for: incremental model updates, measured rollout

Blue-green deployment

- Two identical environments
- Switch 100% of traffic at once
- Old environment stays warm
- Instant rollback: flip the switch

Best for: major version upgrades, hard cutover requirements

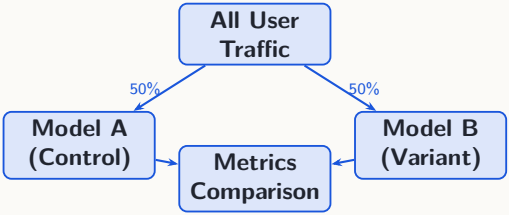
4 A/B Testing as a Deployment Strategy

What A/B testing does

- Different models serve *different user segments*
- Compare business metrics: click rate, conversion, retention
- Randomised assignment ⇒ causal claims about model quality
- Not just for UI changes — essential for **model comparison**

When to use A/B vs. canary

- **Canary**: reduce deployment risk, same model goal
- **A/B**: measure business impact of two model *strategies*
- Often run together: canary → A/B → full rollout



Statistical significance required before declaring a winner.

4 BentoML and Triton Inference Server

BentoML

- Wraps Python models into REST/gRPC services
- Works with sklearn, PyTorch, TensorFlow, XGBoost
- Built-in: model versioning, adaptive batching, Docker packaging
- Deploy locally, on cloud, or on Kubernetes
- Great for: teams that want simplicity without sacrificing power

```
bentoml serve model:svc -port 3000
```

NVIDIA Triton Inference Server

- Optimised inference for GPU and CPU
- Supports: TensorFlow, PyTorch, ONNX, TensorRT
- **Dynamic batching**: groups concurrent requests automatically
- **Concurrent model execution**: multiple models in parallel
- **Model ensembles**: chain models into pipelines

Used at NVIDIA, Microsoft, LinkedIn, and major cloud providers

Other tools in this space

TorchServe (PyTorch-native), Ray Serve (Python-native distributed), TF Serving (TensorFlow-native), Seldon Core (Kubernetes-native)

4 Quiz 2

Q1 — Choose Your Deployment Strategy

Your team has trained a new recommendation model. The old model has been in production for 6 months. You want to validate the new model's business impact (click-through rate) before full rollout. Downtime is not acceptable.

Which deployment strategy (or combination) would you use? Justify your choice.

Q2 — Batch vs. Streaming Features

A ride-sharing app predicts surge pricing every minute. Feature 1: "average ride demand in this area over the past 30 days" (updated nightly). Feature 2: "number of ride requests in this area in the past 5 minutes" (event stream).

Which is a batch feature? Which is a streaming feature? How does the system architecture differ?

4 Quiz 2 — Answers

A1 — Choose Your Deployment Strategy

Recommended: Shadow → Canary → A/B → full rollout.
Shadow first: run new model in parallel, log predictions, compare offline — zero user risk.
Canary (5–10%): catch infrastructure issues (latency, error rates) before full rollout.
A/B test: split 50/50 to measure CTR impact. Declare winner at $p < 0.05$. Old model keeps serving throughout.

A2 — Batch vs. Streaming Features

Batch feature: average ride demand over 30 days. A nightly Spark job writes it to the offline store, then syncs to Redis.
Streaming feature: ride requests in the last 5 minutes. Too narrow for batch — requires Kafka + Flink maintaining a rolling counter per area, written to Redis in real time.
Architecture difference: batch needs data warehouse + Spark + scheduler; streaming needs Kafka + Flink + Redis.

4 What We Covered Today

§1 The Reality of Model Deployment

- 4 deployment myths debunked
- Models degrade without retraining
- Weibo, Netflix, AWS update cadences
- Scale matters even at 100 employees

§2 Batch vs. Online Prediction

- Three prediction modes
- DoorDash: batch + streaming features
- Streaming features $\neq$ online features
- Trade-off table: freshness vs. cost

§3 Latency, Throughput, and SLOs

- Latency $\neq$ throughput; batching paradox
- Always measure p50/p95/p99 — not mean
- Akamai & Booking.com: latency costs money
- Setting SLOs before choosing model size

§4 Serving Patterns and Deployment

- 4 serving patterns and when to use each
- Shadow, canary, blue-green deployment
- A/B testing for model comparison
- BentoML and Triton Inference Server

Thank You!

P99 latency in production: not your laptop benchmark.

Priyansh Saxena

All materials at:

https://www.priyanshsaxena.com/sst-ml-system-design

